

CSCE 713 SOFTWARE SECURITY · MODULE 05 · LECTURE NOTES

Concurrency Defects

Races with security consequences.

MODULE OUTCOMES

- Explain data races and their undefined behaviour.
- Explain atomicity violations above the data race level.
- Explain time-of-check-to-time-of-use.
- Explain why these are hard to find and to fix.
- Identify the security-relevant races in code.

1 Data races

A DATA RACE IS UNDEFINED BEHAVIOUR, NOT MERELY A WRONG VALUE

Two threads accessing the same memory location concurrently, with at least one of them writing, and no synchronisation between them — and in C, C++, and Rust's unsafe subset this is **undefined behaviour** rather than merely an unspecified result. The distinction matters enormously.

So the compiler is entitled to assume it does not happen — and may therefore cache a value in a register across what you believed was a synchronisation point, reorder the accesses, or split a single write into two instructions, none of which your reasoning about the code accounted for.

Which means a torn pointer or a half-updated length field is entirely possible, and that is immediately a memory safety problem (Module 02) rather than a logic one — a pointer half-written is a pointer to somewhere arbitrary.

And the practical consequence is severity: a race on a size field, a reference count, or a state machine variable produces exactly the defects of Modules 02 and 03, arriving by a different route — which is why concurrency gets a module in a software security course rather than being left to the operating systems one.

- A reference count updated without atomic operations — which produces either a double free or a premature free, and is a classic source of use-after-free in reference-counted C++ code.
- Lazy initialisation without proper synchronisation — the double-checked locking pattern, which is simply wrong without the correct memory ordering and was wrong in published textbooks for years before the memory models were formalised.

- **A shared cache or scratch buffer reused across requests**, which **leaks one user's data to another** — a confidentiality defect arising directly from a concurrency bug, and the one with the most surprising impact because **it looks like a performance optimisation** in review.
- **Signal handlers touching non-atomic state**, which is a race with yourself rather than with another thread — and which is why the set of functions safe to call from a handler is so small.
- **And fork in a threaded program**, which produces a child inheriting locks in arbitrary states, held by threads that no longer exist — a deadlock or a corrupted invariant at unpredictable times.

2 Atomicity violations

EVERY INDIVIDUAL ACCESS CAN BE SYNCHRONISED AND THE SEQUENCE STILL BE WRONG

Check the balance while holding a lock, release the lock, then deduct the amount while holding the lock again — there is no data race anywhere, every access is properly synchronised, and the balance may have changed in between the two.

So the invariant spans more than one operation, which means **the lock has to span the invariant rather than each individual access** — and that is a design question that no automated race detector can ask, because it does not know what the invariant was supposed to be.

And this is the security-relevant concurrency case in managed languages, where **data races are prevented by the runtime and atomicity violations are not** — so a Java or Go service has no torn values and every double-spend defect available.

Which makes it the harder class to find: ThreadSanitizer finds data races very effectively and has no idea what your invariant was (Module 09 §3) — so this class is found by reasoning and by review (Module 11) rather than by tooling.

```
CHECK-THEN-ACT
    if (balance >= amount)    // lock held, released
        balance -= amount;    // lock taken again
    The gap between them is the defect.

READ-MODIFY-WRITE
    x = get(); x++; set(x);
    Two concurrent updates interleave; one is lost.

DOUBLE SPEND / DOUBLE REDEEM
    the same voucher, request, or token processed twice
    because the "already used" check and the marking as
    used are not atomic. Very common in web
    applications, and a direct financial defect.

AND THE FIXES
    hold the lock across the whole invariant
    or use a single atomic operation (compare-and-swap)
    or make the operation idempotent, so that a repeat
        is harmless (CSCE 678 Module 07)
    or enforce uniqueness in the database with a
        constraint, which is atomic for free
```

The database constraint is the underrated fix. A unique index on the redemption record makes double redemption impossible without any application-level locking at all — the second insert fails, the transaction rolls back, and the invariant is enforced by the one component that is already designed to do exactly this. **Practitioners reliably reach for a lock and forget the constraint**, which is both more complex and less reliable across multiple application instances (CSCE 678 Module 04).

3 Time-of-check-to-time-of-use

A PROPERTY CHECKED AND THEN RELIED UPON MAY CHANGE IN BETWEEN

Check that a path is not a symbolic link, then open it — and an adversary who can replace the path between those two operations defeats the check entirely, because the check examined a different object than the open received.

Which generalises well beyond the filesystem: check a permission then act on it, check a quota then allocate, validate a URL then fetch it, verify a file's hash then execute it — all exactly the same shape, and all defeated by a change in between.

And the fix is to make the check and the use a single operation — open the file and then inspect the resulting descriptor, use `openat` with `O_NOFOLLOW` and the appropriate flags, or hold a lock across both operations so that no change can intervene.

The general principle: operate on the handle, not on the name — because the name is a lookup that can resolve differently the next time it is performed, and the handle is the actual thing you checked. Which is the same reasoning as Module 04 §3's resolve-then-contain, and the two modules' advice composes: resolve, open, then verify the descriptor.

4 Why they are hard, and what helps

- They are non-deterministic, so a test that passes proves nothing at all and a failure may not reproduce on demand — which breaks the normal debugging loop entirely.
- They are timing-dependent, so they appear under production load and on different hardware rather than on a developer's machine — which means they are found by users and by incidents.
- Fuzzing finds them poorly, because the input space that matters is the *interleaving* rather than the data (Module 08 §4), and a conventional fuzzer varies only the data.
- And static analysis finds them poorly, because it requires sound reasoning about all possible interleavings — which is where CSCE 627's undecidability results bite hardest, and why concurrency analysers have either very high false positive rates or very narrow scope.
- What helps: ThreadSanitizer for the data races, stress testing with artificially injected delays at suspicious points, and reducing shared mutable state so that there is less to get wrong. Reducing shared mutable state is the structural answer — message passing and immutability remove the class rather than detecting instances of it, which is the same move as Module 02 §4's and is available in any language.

AND WHAT A SAFE LANGUAGE DOES AND DOES NOT PREVENT

Rust's ownership and borrowing system prevents data races in safe code — which is a real and genuinely unusual guarantee, not shared by the managed languages — and **it covers §1 entirely**.

It does not prevent atomicity violations, deadlocks, or time-of-check-to-time-of-use defects — those are logic defects, and the type system does not know what your invariant was. A Rust program can double-spend exactly as readily as a C one.

And the managed languages prevent torn values and therefore the memory-unsafe consequences of §1, **while leaving every logical race entirely intact** — which is a weaker guarantee than Rust's and still removes the severe cases.

So the honest claim is 'no data races in safe code', and §2 and §3 remain your problem — which is the same shape of claim as Module 02 §4's and deserves the same precision, because the broad version ('Rust is safe') leads teams to stop thinking about the classes that remain.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Boehm — Threads Cannot Be Implemented As a Library (free)	§1's argument for why the memory model must be in the language — and why a data race is undefined rather than merely unlucky.
Lu et al. — Learning from Mistakes: a study of real-world concurrency bugs (free)	§2's classification, measured — the study that established atomicity violations as the dominant real-world class.
ThreadSanitizer documentation (free)	§4's primary tool — what it detects, what it misses, and its cost.
CWE-362, CWE-367, CWE-416 (free)	§2 and §3 with the shared vocabulary — race condition, TOCTTOU, and use-after-free.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write a non-atomic reference count** and produce a double free under load.
- 2 **Catch it with ThreadSanitizer** and read the report.
- 3 **Write a shared scratch buffer** and demonstrate cross-request data leakage.
- 4 **Write a check-then-act double spend** and exploit it with concurrent requests.
- 5 **Fix it four ways:** wider lock, compare-and-swap, idempotency, and a unique constraint.

- 6 **Compare the four fixes** for complexity and for behaviour across multiple instances.
- 7 **Write a TOCTTOU against your own test directory** using a symlink swap.
- 8 **Fix it by operating on the descriptor.**
- 9 **Try fuzzing a racy program** and report whether the race was found.
- 10 **State precisely what Rust prevents** and what it does not.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is a data race undefined behaviour, and what may the compiler do?
- 2 How does a race become a memory safety defect?
- 3 Name five patterns that produce data races.
- 4 Why is the shared-buffer case surprising?
- 5 What is an atomicity violation, and why does ThreadSanitizer miss it?
- 6 Give three shapes and four fixes.
- 7 Why is the database constraint underrated?
- 8 Explain TOCTTOU and give the general principle.
- 9 Give four reasons these defects resist the usual methods.
- 10 State precisely what a safe language prevents here.